

Implantación de sistemas seguros de despliegado de software.

Caso práctico



[Pixabay](#) (Dominio público)

Julián ha empezado un nuevo trabajo en el departamento de DevOps de una empresa de Ciberseguridad. Dicho departamento trabajará creando aplicaciones y desarrollos junto a los analistas de seguridad del SOC (Centro de Operaciones de Seguridad). El modo de trabajo será de entregas continuas en vez de esperar a que el proyecto esté finalizado. Al estar junto al consumidor de estas aplicaciones el feedback será continuo entre creador y consumidor y podrán pulirse muchas situaciones antes de llegar a producción, siendo un beneficio para los desarrolladores y los clientes o consumidores.

El ciclo de desarrollo de software ha cambiado mucho durante los últimos años, nuevos modelos como DevOps con entregas y alineamientos continuos con las necesidades del cliente han cambiado la metodología e incluso la cultura en el desarrollo. El modelo ha mutado hacia una comunicación continuación el cliente, basado en entregas cortas y un continuo alineamiento con las necesidades reales. Esto ha provocado que el resultado se acerque más a las expectativas del cliente y se pueda cubrir cualquier desviación con mas agilidad.

Por otra parte tenemos que garantizar que cuando implantemos nuestro software sea de la

manera más segura y colaborativa posible, garantizando que todos los miembros del equipo pueden escribir su código y que tenemos disponible un control de versiones, un entorno seguro de desarrollo, etc



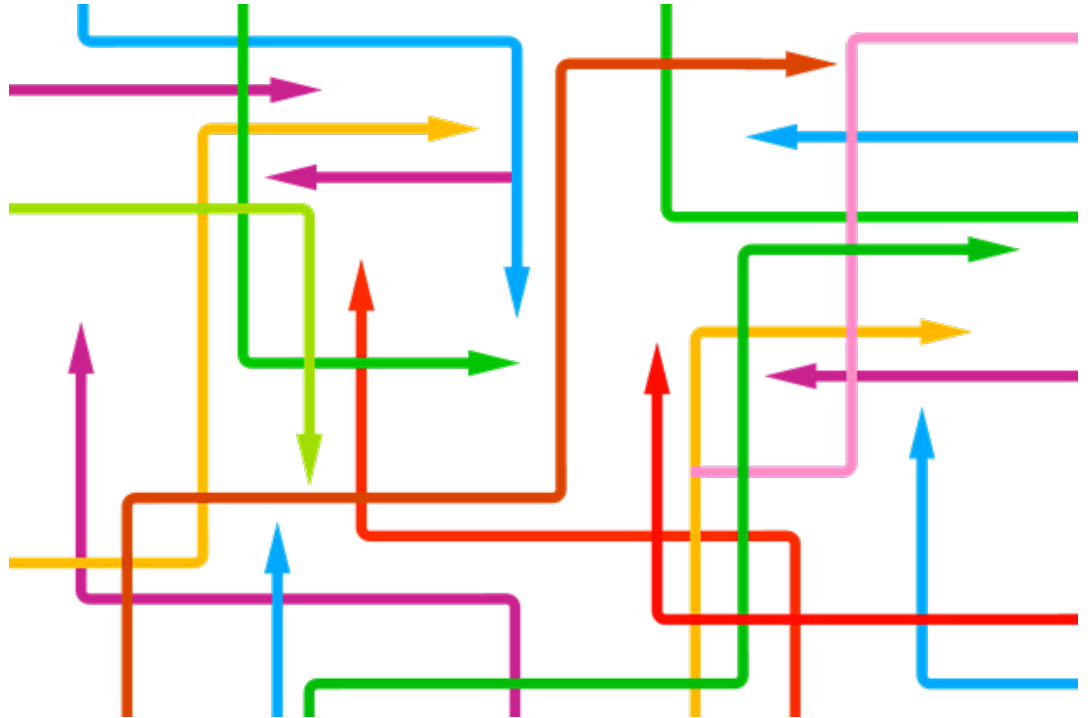
[Ministerio de Educación y Formación Profesional](#) (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#)

1.1.- Puesta Segura en producción y DevOps.

La puesta segura en producción del código que producen los desarrolladores ha evolucionado mucho durante los últimos años, ahora los desarrollos son en equipo (es decir, todos los desarrolladores conocen en tiempo real el desarrollo exacto que está haciendo el compañero) y muy cercano al cliente. A raíz de



[Pixabay](#) (Dominio público)

esto uno de los conceptos que hoy en día se usa con mayor asiduidad es el concepto de **DevOps**, por haber supuesto un cambio de paradigma en el modelo tradicional de producción de software.

DevOps no es más que una **metodología** basada en la creación de software de forma continua y de mayor calidad mediante versiones mas frecuentes. Esto se traduce de forma directa en claras ventajas:

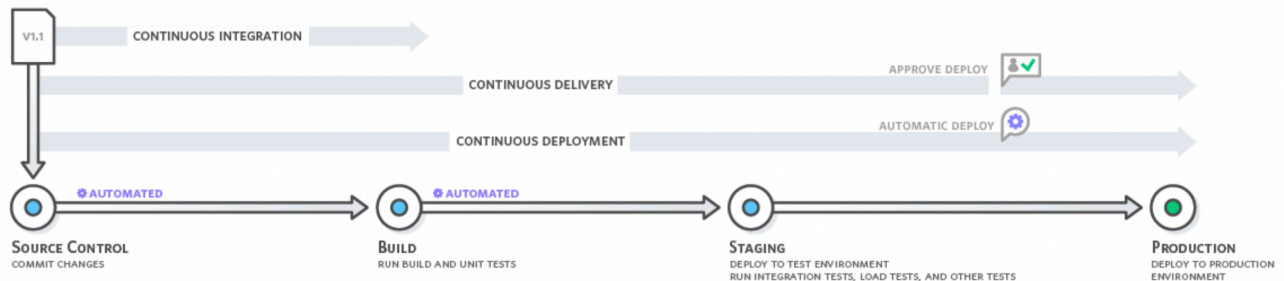
- ✓ Las versiones mas frecuentes hace que se pueda alinear la expectativa del cliente de forma más sencilla.
- ✓ Cualquier cambio en el proyecto se puede corregir con más facilidad.
- ✓ El software tiene más calidad al estar enfocado en objetivos más concretos.

Para saber más

Durante los últimos años ha surgido junto a la metodología de DevOps nuevas maneras de gestionar proyectos y entregas con clientes basadas en

metodologías ágiles (*agile* en inglés). Si bien ambas tienen algunos principios similares y pueden ser complementarias es conveniente saber las diferencias y puntos de convergencia. Te dejamos [este enlace](#) si quieres saber más sobre el tema.

1.2.- Control de versiones e Integración Continua.



[Amazon AWS](#) (Captura de pantalla)

Como comentábamos uno de los principales cambios que surgen con la adopción de la metodología de DevOps es el continuo desarrollo de software mediante lanzamiento o **releases** continuos.

Si a todo este proceso de nuevas versiones y lanzamientos unimos a varios miembros del equipo editando, codificando y generando cambios en las distintas piezas del software, hace que sea fundamental poder disponer de un modelo automático de control de versiones y cambios.

Los equipos de desarrollo trabajan con sistemas de control de versiones (o VCS, por sus siglas en inglés), que son herramientas software que gestionan, administran y controlan (creación, adición, eliminación, modificación del código) sobre un **repositorio** común.

Este repositorio esta disponible para todos los desarrolladores y ofrece una manera colaborativa de trabajar donde todos los programadores pueden tener acceso al código y conocer qué cambios se han hecho, quien y cuando los ha hecho y el estado actual de la versión. También proporciona una manera sencilla de tener un **backup del código y recuperarnos** de algún cambio erróneo o problema. Este repositorio es el resultado visible de lo que se conoce como **Integración continua**.

La integración continua es una práctica de desarrollo de software mediante la cual los desarrolladores combinan los cambios en el código en un repositorio central de forma periódica, tras lo cual se ejecutan versiones y pruebas automáticas.

Las ventajas de ambos son:

Control de Versiones

- ✓ Revertir y corregir cambios de manera sencilla.
- ✓ Copia de seguridad o backup del código.
- ✓ Resolución de conflictos cuando uno o más miembros modifiquen el mismo fichero a la vez.

Integración Continua

- ✓ Realización de pruebas que solucionen errores de manera automática.
- ✓ Entregar más rápido las versiones al cliente.
- ✓ Mejorar la productividad al estar los desarrolladores centrados solamente en desarrollar.

Debes conocer

La integración continua está estrechamente relacionada con el control de versiones, los desarrolladores envían los cambios sobre el software de forma periódica al repositorio compartido con un sistema de control de versiones. Ambas soluciones se aportan por un lado la integración continua proporciona un repositorio común que se mantiene actualizado y el control de versión asegura tener todos los cambios controlados.

Uno de los sistemas de control de versiones más famosos y que se ha convertido en el estándar es **Git**, es un sistema de control de versiones distribuido usado por los desarrolladores de software. Tienes mas info en este enlace de [wikipedia](https://es.wikipedia.org/wiki/Git).

En Git hay tres etapas primarias (condiciones) en las cuales un archivo puede estar: estado **modificado**, estado **preparado**, o estado **confirmado**.

No hay confundir **Git** con **Github**, este último es una plataforma basada en la web donde los usuarios pueden alojar repositorios Git. Facilita compartir y colaborar fácilmente en proyectos con cualquier persona en cualquier momento.

Mediante Git podremos **clonar un repositorio** o incluso si queremos probar el software podremos **construirlo (build)** de manera automática usando el mismo código que han usado los desarrolladores para crearlo.

Autoevaluación

Identifica si las siguientes frases son verdaderas o falsas

DevOps es un nuevo estándar de programación.

☐ Verdadero ☐ Falso

Falso

DevOps es una metodología de desarrollo que aporta numerosos beneficios.

Hoy en día los desarrolladores funcionan de forma aislada.

☐ Verdadero ☐ Falso

Falso

Los desarrolladores suelen trabajar en grupo en la creación de software.

DevOps ha cambiado la manera en que se entrega el software al cliente.

☐ Verdadero ☐ Falso

Verdadero

La metodología de DevOps mejora la calidad del software mediante entregas más rápidas al cliente.

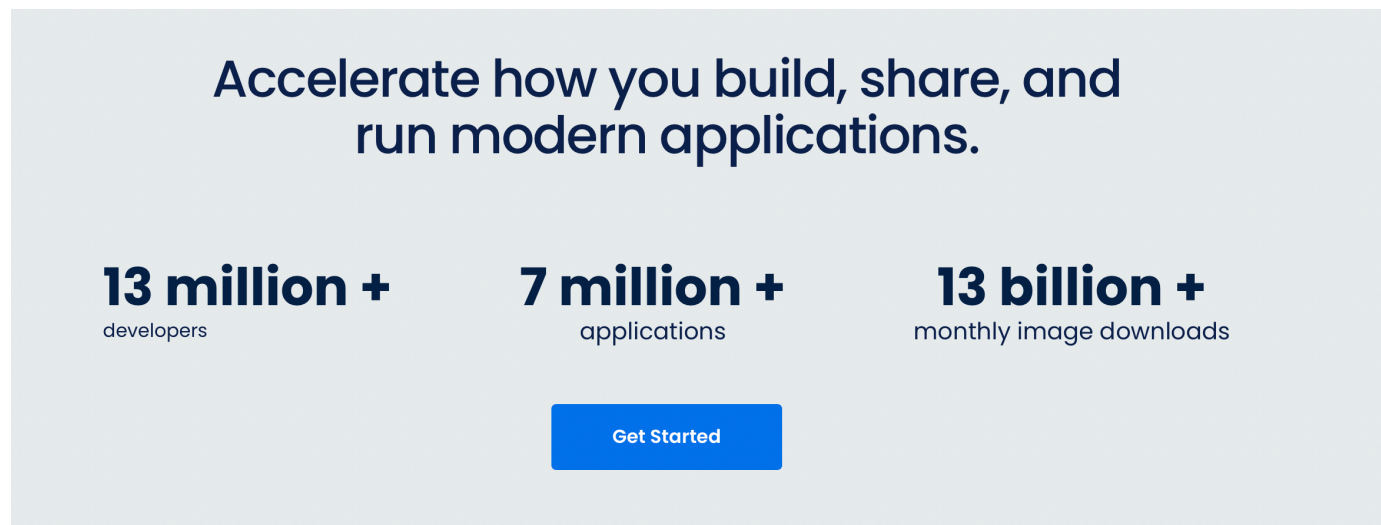
Para que todos los desarrolladores trabajen de forma colaborativa y eficiente surgen los repositorios.

☐ Verdadero ☐ Falso

Verdadero

Los repositorios son la manera estándar de almacenar el código y trabajar de forma colaborativa.

1.3.- Virtualización y contenedores.



[Docker](#) (Captura de pantalla)

Uno de los problemas que se presentan durante el desarrollo y despliegue de aplicaciones es la heterogeneidad entre los sistemas de desarrollo, no ya entre los propios desarrolladores, también entre los sistemas de desarrollo, entornos de pruebas, pre-producción y producción. Los sistemas donde se crea y se prueba el software tienen librerías y sistemas operativos diferentes. Esto provoca que la aplicación o software podría funcionar bien con unas determinadas versiones de librerías o sistema operativo pero no en otros, o ejecutarse bien con la versión de un lenguaje pero no con otra. Para asegurar la calidad de desarrollo tenemos que asegurar que todo el equipo de desarrollo usa las mismas versiones de todas las aplicaciones y librerías necesarios.

Esto es más complicado de lo que parece, porque hay desarrolladores que prefieren una distribución concreta, o incluso sistemas privativos. Incluso los sistemas de pruebas, pre-producción y producción suelen ser distintos. Los sistemas de producción suelen ser más nuevos y potentes y los antiguos se dejan para pruebas y pre-producción. Otro problema derivado aún más importante es que los equipos desarrollo a veces están involucrados en varios proyectos a la vez y cada uno de estos proyectos requiera versiones distintas de librerías, complicándolo aún más. De hecho podría pasar que cambios que se hagan en una librería que afecta a un proyecto acabe afectando sin querer a otro proyecto, suponiendo un gran problema.

Para solucionar estos problemas es importante que los desarrolladores creen el código en entornos controlados y fácilmente replicables. Al principio los desarrolladores optaron por usar máquinas virtuales para solventar estos problemas pero hoy en día el estándar es el uso de contenedores, que son entornos (con sistema operativo, librerías de software, entorno de desarrollo de software, etc) ya creados y prefijados con una configuración específica que simplifican el proceso. Se pueden exportar y usar en cuestión de segundos y hace que todo el equipo de desarrollo trabaje con las mismas herramientas, y en el caso de que estén en varios proyectos usan contenedores distintos. Durante los últimos años una de las referencias a nivel de contenedores y que se considera casi el estándar ha sido **Docker**.

Los contenedores también crean nuevas necesidades, como gestionar la escalabilidad. A medida que crece el proyecto se hace difícil gestionar el ciclo de vida del contenedor y su administración. Para solucionar este problema, surge la orquestación de los contenedores. Mediante esta práctica se resuelve el problema automatizando la programación, implementación, escalabilidad, equilibrio de carga, disponibilidad y redes de contenedores. La orquestación de contenedores no es más que es la automatización y gestión del ciclo de vida de contenedores y servicios. Una de las soluciones mas conocidas de orquestación de contenedores es **Kubernetes**.

Debes conocer

Kubernetes es una plataforma de código abierto que fue diseñada y creada por Google. Kubernetes ayuda a automatizar la implementación, el escalado y la administración de cargas de trabajo y servicios en los contenedores.

Por otra parte puedes conocer más sobre la tecnología Docker y sus numerosos beneficios en este [enlace](#).

1.4.- Desarrollo Ágil.



[Agile Manifesto](#) (Captura de Pantalla)

Como comentábamos la manera de desarrollar software ha cambiado, necesitamos adaptarnos de forma sencilla ante los cambios del cliente y para poder hacerlo surgen las metodologías ágiles en el desarrollo. Estas metodologías se basan en una serie de principios básicos:

- 1.- Satisfacción del cliente mediante la entrega temprana y continua de software de calidad.
- 2.- Gran aceptación y respuesta al cambio de los requisitos definidos durante el proyecto.
- 3.- Entrega frecuente de software funcional, periodos 2-6 semanas.
- 4.- Trabajo conjunto entre el cliente y desarrolladores.
- 5.- Creación de equipos motivados y respaldados.
- 6.- Comunicación cara a cara desarrolladores-cliente.
- 7.- Equipos auto-organizados.
- 8.- Reflexión continua de formas para mejorar la efectividad y realizar los ajustes necesarios para corregir cualquier desviación.

De entre todas ellas surge **Scrum**, una metodología ágil que engloba una serie de principios que ayudan a los equipos a desarrollar productos en periodos cortos (llamados sprints), permitiendo obtener de forma rápida feedback, adaptaciones y una mejora continua.

Dentro de los Sprints también se hacen ciclos de verificación o bucles, y simulación de fallos.

Estos bucles ayudan a verificar la calidad del software y tomar las medidas oportunas sino fuese así, ayudando a mejorar la calidad del software. Además estos bucles retroalimentan el proceso, mejorando la calidad del producto final.

Debes conocer

Existe un manifiesto que marca los principios fundamentales de la metodología ágil en el desarrollo de software. No especifica el cómo pero si define los principios fundamentales. Puedes consultarlo [aquí](#).

Para saber más

Unos de los bucles o ciclos más famosos es el ciclo Deming, es famoso por ser el que llevó a Toyota a convertirse en uno de los gigantes de la industria del automóvil y a Japón ser un referente de la industria. Se utiliza también dentro de la metodología Scrum como un proceso de mejora continua. Puedes consultar más info [aquí](#).

